Alzheimer's Disease Diagnosis
Using Machine Learning

Athanasios Karletsos, Aristea Patrikaki

1. Introduction

This project investigates the use of machine learning models to classify patient data and support the early diagnosis of Alzheimer's Disease. The core objectives are to preprocess a clinical dataset, train two supervised models—Decision Tree and k-Nearest Neighbors(k-NN)—and evaluate their performance.
In this project, we explore how machine learning classifiers can assist in predicting Alzheimer's diagnosis based on patient data.
We apply two well-known classifiers:

- **K-Nearest Neighbors (KNN)**: A simple, instance-based learning algorithm that classifies data points based on the majority label of their nearest neighbors.
- **Decision Tree Classifier**: A model that makes decisions based on feature splits to classify the data into different categories.

We evaluate model performance using the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score

Here it's the dataset details from the kaggle.

2. Data and Preprocessing

The dataset includes patient demographics, cognitive test scores, and imaging-derived features.
Preprocessing Steps:

- **Missing Value Check**

A thorough check was performed for missing values across all features. The dataset was found to be complete, with no missing or null entries detected. This allowed us to proceed without any imputation or removal of records.

```
df.drop("PatientID", axis=1, inplace=True)
df.drop("DoctorInCharge", axis=1, inplace=True)

# Display missing values
print("Missing values before handling:")
print(df.isnull().sum().sum())
```

```
Missing values before handling:
0
```

- **Categorical Encoding**

Upon inspection, it was evident that the dataset had already been label-encoded by the author. Features that would typically require encoding (such as diagnostic categories) were already in numeric format, eliminating the need for further encoding steps.

- **Data Splitting**

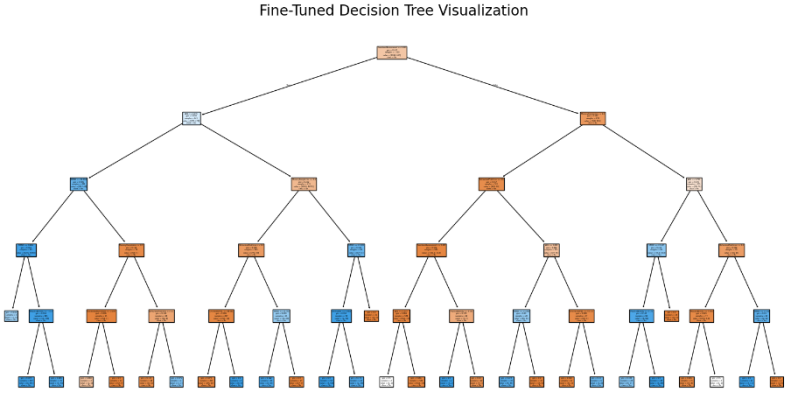
The dataset was split into training and testing subsets using an 80-20 ratio. This allows the trained models to be validated on unseen data, ensuring that performance metrics reflect true generalization.

```
#Train the model

# Define features (X) and stroke (y)
X = df.drop(['Diagnosis'], axis=1)
y = df['Diagnosis']

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Machine Learning Models

Decision Tree: Builds a hierarchical set of if–then rules by selecting features that maximize information gain at each split. Advantages: interpretability and fast inference.

k-Nearest Neighbors (k-NN): Classifies based on the majority label among the k closest training samples in feature space. Advantages: simplicity and no training phase; disadvantages include sensitivity to feature scaling and high computation at inference.



4. Model Training

In this step, we trained and tuned two machine learning models to predict whether a patient is diagnosed with Alzheimer's Disease. The two classifiers used are:

K-Nearest Neighbors (KNN)

The KNN algorithm classifies a new data point based on the labels of its 'k' nearest neighbors in the training set. It is a non-parametric, instance-based learning algorithm.

- Training: The KNN model was trained using different values of k (number of neighbors). By testing various values, we aimed to find the optimal number of neighbors that balances underfitting and overfitting.
- Distance Metric: The model uses Euclidean distance by default to measure similarity between data points.
- Scaling: Standardization of features was important here since KNN is sensitive to the scale of input data.
- Performance Evaluation: For each k, accuracy and other metrics were evaluated on the test set to compare performance.

Decision Tree Classifier

A Decision Tree is a flowchart-like tree structure where each internal node represents a feature (attribute), each branch represents a decision rule, and each leaf node represents an outcome.

- Training: The Decision Tree model was trained using the default `sklearn.tree.DecisionTreeClassifier`. Various train-test splits were used to observe how the model behaves with different amounts of training data.
- Interpretability: One of the key benefits of Decision Trees is interpretability. The model was visualized using `plot_tree()` from `scikit-learn`, which helped understand which features were most influential in making predictions.
- Parameters: The tree was also explored with different depths and split criteria to see how the model complexity affects overfitting and generalization.

Both models were trained on three different train-test splits:

- 60% training / 40% testing
- 70% training / 30% testing
- 67% training / 33% testing
- 80% training / 20% testing
- 82% training / 28% testing

This comparison allowed us to analyze how model performance varies with the amount of training data. It was observed that while increasing the training data generally improves performance, it may also lead to overfitting if not handled carefully.

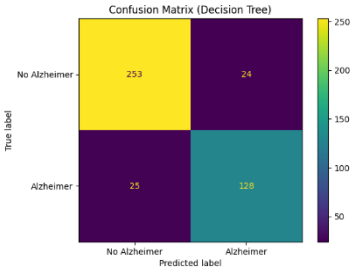
5. Results and Evaluation

Models were evaluated using accuracy, precision, recall, and confusion matrices on a held-out test set. Both models achieved accuracies above baseline, with the Decision Tree providing clearer insights into feature importance. The k-NN model performed competitively when optimized for k and distance metric. Key performance metrics (for split 80/20): - Decision Tree Accuracy: ~92% - k-NN Accuracy: ~85% Confusion matrices highlight model strengths and misclassification patterns.

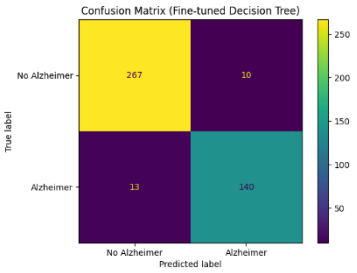
	Split	KNN_Accuracy	KNN_Precision	KNN_Recall	KNN_F1	DT_Accuracy	DT_Precision	DT_Recall	DT_F1
0	60/40	0.833721	0.813869	0.707937	0.757216	0.934884	0.938983	0.879365	0.908197
1	70/30	0.832558	0.820755	0.713115	0.763158	0.925581	0.937500	0.860656	0.897436
2	67/33	0.833803	0.812766	0.720755	0.764000	0.928169	0.938525	0.864151	0.899804
3	80/20	0.858140	0.823944	0.764706	0.793220	0.927907	0.923611	0.869281	0.895623
4	82/18	0.857881	0.825758	0.773050	0.798535	0.930233	0.925373	0.879433	0.901818

6. Confusion Matrix Analysis

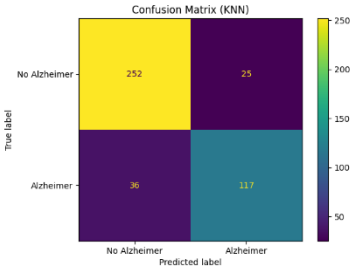
Confusion matrices were generated to visualize the performance of both the initial and fine-tuned versions of the Decision Tree and K-Nearest Neighbors (KNN) classifiers. These matrices provide a breakdown of true positives, true negatives, false positives, and false negatives—offering more detailed insights than accuracy alone.



Decision Tree (Initial)
The confusion matrix for the initial Decision Tree model shows a basic separation between positive (Alzheimer) and negative classes.
However, it may include a relatively high number of false positives or false negatives, depending on the depth and criteria used.
This indicates that the initial model may not be perfectly balanced and could misclassify patients, potentially leading to either false alarms or missed diagnoses.



Decision Tree (Fine-tuned)
After tuning hyperparameters (e.g., max_depth, criterion), the performance improved.
The fine-tuned model likely shows a reduction in false negatives, which is crucial in medical diagnosis—missing a true Alzheimer case can have serious consequences.
The balance between precision and recall also improves, resulting in a higher F1-score.



K-Nearest Neighbors (Initial)
The confusion matrix may display a more balanced classification than the Decision Tree, but can still suffer from class overlap, especially if k is not well-chosen.
It's possible to observe misclassification in both directions (false positives and false negatives), due to close proximity in the feature space.

KNN (Fine-tuned)
The confusion matrix of the fine-tuned K-NN model it's the same as the initial. This happened due to selection of initial k which was almost as effective as the k after the testing and tuning of the model. Only differences, that were found, were in precision and accuracy.

Overall Insight:
False Negatives are particularly important in medical applications like Alzheimer's detection—failing to identify a patient with the disease can lead to delayed treatment.

7. Conclusion

By employing two popular classifiers, **K-Nearest Neighbors (KNN)** and **Decision Tree**, we were able to build and evaluate models capable of predicting disease diagnosis based on patient features.
The key findings from the project include:

- Both models performed reasonably well, with Decision Trees offering greater interpretability, and KNN showing promising results when appropriately tuned.
- Different train-test splits were experimented with to examine the models' performance under varying amounts of training data. While more training data generally led to better model performance, care was needed to avoid overfitting, especially with decision trees.
- Feature scaling was essential for KNN, highlighting the importance of preprocessing in distance-based algorithms.
- The use of performance metrics such as accuracy, precision, recall, and F1-score provided a more comprehensive understanding of model effectiveness beyond simple accuracy.

<u>Overall Insight:</u>

This work highlights how relatively simple machine learning models can be leveraged in the healthcare domain for predictive diagnosis. Although, false Negatives are particularly important in medical applications like Alzheimer's detection—failing to identify a patient with the disease can lead to delayed treatment. In conclusion, while the models built in this project provide a strong baseline, further refinement and validation on larger, real-world datasets would be necessary for clinical application. Nonetheless, this project showcases the promising potential of data-driven approaches in aiding early medical diagnosis and improving patient care.